

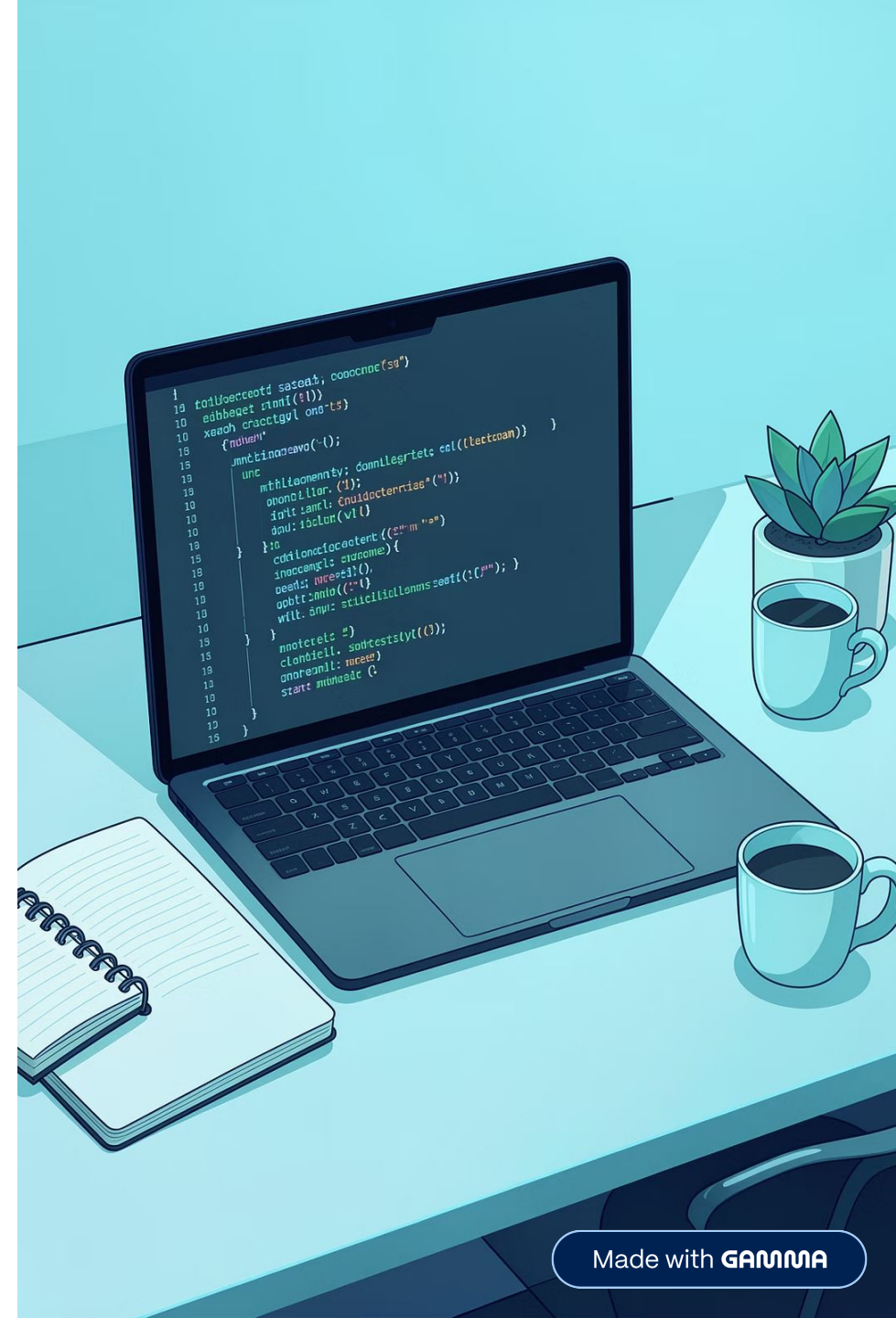
Python & Jupyter Notebooks: First-Time Desktop Setup Guide

Install Python, choose your IDE, set up a virtual environment, and run your first notebook — complete beginner guide.

 BEGINNER FRIENDLY

 IDE AGNOSTIC

 SELF-PACED





The Big Picture: Your End-to-End Workflow



Desktop

Your computer workstation.



Python Engine

The core language runtime. This component executes your Python code, making your scripts and applications run.



Virtual Environment (.venv)

Isolated project sandbox. Manages project-specific dependencies to prevent conflicts between different projects.



Modern IDE (Antigravity / VS Code)

Your code workbench. Offers comprehensive tools for coding, debugging, and project management, enhancing productivity.



Jupyter Notebook (.ipynb)

Interactive document with live code & outputs. Combines code, output, and explanatory text for interactive development and data analysis.

These 5 components work together to create a complete Python development environment. You'll set them up in order, and by the end, you'll have a fully functional notebook ready for coding.

Choose Your Setup Path

Pick the workflow that fits your comfort level before you start building.



Path A — Antigravity (Recommended)

- Single window, no server setup
- Built-in AI agent for error fixing
- Google account required (free)
- Best for: Beginners, self-paced learning
- No kernel selection issues

[Download Antigravity IDE](#)[Open in Browser](#)

Path B — VS Code + Terminal

- Traditional terminal + IDE workflow
- GitHub Student account required (free with .edu email)
- More control, steeper learning curve
- Best for: Those wanting full control
- Requires kernel selection

[Download VS Code](#)[Open Website](#)

Step 0: Get Your Accounts Ready

Path A — Antigravity

Sign up for a free Google account (if you don't have one already). This is your gateway to Antigravity IDE and its AI agents.

Path B — VS Code

Get a GitHub Student account at github.com/student. Use your **.edu** email. This unlocks GitHub Copilot (AI in VS Code) and other student benefits.

⚠️ Account setup is non-negotiable — your AI agent depends on it!

Python Installation

Download Python 3.12

1 Download Python

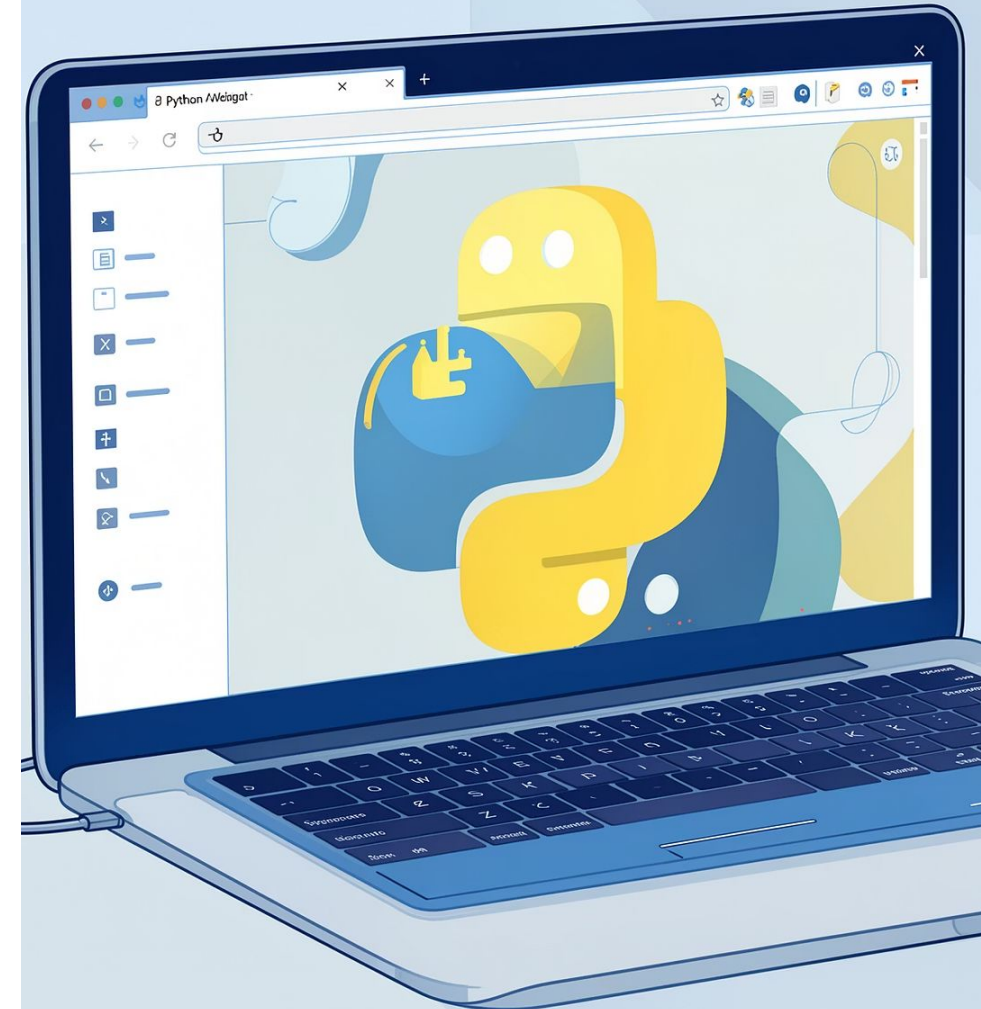
Go to python.org and download Python 3.12 from the link above.

2 Run the Installer

On **Windows**: tick **"Add Python to PATH"** before clicking Install Now.

3 Verify in Terminal

Open a terminal and run `python --version`.



How AI Agents Transform Your Coding Workflow

What Are AI Agents in Your IDE?

-  Real-time error detection and fixes
-  Code suggestions as you type
-  Instant debugging explanations
-  Natural language to code conversion

Example: "Write a function to calculate Black-Scholes pricing" → AI generates the code

Why This Matters for Any Coder:

-  Complex logic is hard — AI catches syntax errors instantly
-  Debugging takes forever — AI validates your logic in seconds
-  Learning by doing — AI explains what went wrong and why
-  No more "kernel selection" trap — Antigravity handles it
-  Faster iteration: write → test → fix in seconds, not minutes



Pro Tip: Your AI agent is like having a senior developer sitting next to you. Ask it questions, let it catch your mistakes, and focus on your logic.

Step 1: Install Your IDE & Create

Download & Install

[Download Antigravity IDE](#)

[Download VS Code](#)

Download your chosen IDE

Use one of the links above

Run the installer

Windows: .exe • Mac: drag to Applications

Launch and sign in

Use your account to get started

Create a new project

Name it **python-setup**

Success checkpoint

You see the editor with the sidebar

First Look at the AI Agent

Click the AI icon

Find the sparkle/star in the sidebar

Send your first message

"Hello, I'm learning Python. Can you help me set up my environment?"

Get guidance

The AI agent responds with context-aware suggestions

Use it throughout the course

This is your debugging partner for the entire course

☐ Your IDE handles all the complexity for you. The AI agent is always one click away.

Step 2: Create a Virtual Environment (.venv)

Why Virtual Environments?

Isolation

Each project gets its own packages

No conflicts

Changes in one project never break another

Clean

Easy to delete and start fresh

Professional

This is how real developers work

Create Your .venv

Windows

```
python -m venv .venv  
.venv\Scripts\activate
```

macOS/Linux

```
python3 -m venv .venv  
source .venv/bin/activate
```

Success checkpoint: Look for **(.venv)** in green at the start of your terminal prompt.

📌 Your virtual environment is now active. All packages you install will go into this isolated folder, not your system Python.

Step 3: Install Essential Packages

Install Packages

```
pip install numpy pandas matplotlib jupyter
```

With your **(.venv)** active, run this command in your terminal.

Success checkpoint

You should see '**Successfully installed**' with no red errors.

These are the foundation packages for data science and Python development. Your AI agent can explain what each one does if you ask.

What Each Package Does

- **numpy**: Fast numerical computing
- **pandas**: DataFrames for organizing data
- **matplotlib**: Create charts and visualizations
- **jupyter**: Interactive notebooks

Step 4: Create Your First Notebook

Create a New Notebook

- In your IDE, click "New File" → "Notebook"
- Name it: **first-notebook.ipynb**
- You're now in a blank notebook with cells ready for code

Your First Cell (Copy-Paste Ready)

```
import numpy as np
import matplotlib.pyplot as plt

# Create some data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Plot it
plt.plot(x, y)
plt.title("My First Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Paste this into your first cell and press Shift+Enter to run it. You should see a sine wave chart.

 If something breaks, ask your AI agent: "Why did this error occur?" It will explain and suggest fixes.

Step 5: Understanding Cells & Kernels

SECTION 1 - What Are Cells?

- A cell is a block of code or markdown text
- You write code in one cell, run it with Shift+Enter
- Results appear directly below
- You can edit and re-run cells as many times as you want

SECTION 2 - What's a Kernel?

- The kernel is the Python engine running your code
- In Antigravity, the kernel is pre-configured — no selection needed
- In VS Code, you may need to select the kernel (this is where beginners get stuck)
- Antigravity advantage: One less thing to worry about

SECTION 3 - Your First Workflow

- Write code in a cell
- Press Shift+Enter to run
- See output below
- Edit and re-run instantly
- Ask your AI agent if something breaks



This is why Antigravity is recommended — no kernel selection trap, no server crashes, just code and results.

Step 6: Working with Data in Notebooks

SECTION 1 - Load and Explore Data:

```
import pandas as pd

# Create a simple dataset
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Score': [85, 90, 78]
}

df = pd.DataFrame(data)
print(df)
print(df.describe())
```

SECTION 2 - What This Means:

- You created a DataFrame (a table of data)
- You printed the data
- You used .describe() to get statistics
- This is the foundation of data analysis

 DataFrames are the core of data science. Your AI agent can help you filter, group, and analyze data.

Setup Complete — Your Next Steps

Verification Checklist

-  Account created (Google or GitHub)
-  IDE downloaded and installed
-  Python installed and verified
-  Virtual environment (.venv) created and activated
-  Essential packages installed (numpy, pandas, matplotlib, jupyter)
-  First notebook created
-  First plot displayed
-  Data exploration completed

What's Next?

1. Master Python fundamentals (variables, functions, loops)
2. Learn data science with pandas and matplotlib
3. Build your first real project (weather app, data analyzer)
4. Explore advanced libraries (scikit-learn, requests, beautifulsoup)
5. Contribute to open source on GitHub
6. Keep learning — your AI agent is always there to help

  Congratulations! You've set up a professional Python development environment. You're now ready to start your coding journey. Remember: every expert was once a beginner. Ask your AI agent questions, build projects, and enjoy the learning process. Happy coding!

Managing Dependencies with requirements.txt

What is requirements.txt?

- A simple text file that lists all packages your project needs
- Makes it easy to share your project with others
- Allows others to install the exact same packages you used

Example content:

```
numpy==1.24.3
pandas==2.0.2
matplotlib==3.7.1
jupyter==1.0.0
```

How to Use It

1. After installing packages in your .venv, run:

```
pip freeze > requirements.txt
```

1. This creates a file with all your installed packages
2. To share your project: Send the folder (without .venv)
3. Others can recreate your environment with:

```
pip install -r requirements.txt
```

 This is how professional developers share code. It ensures everyone has the same package versions, preventing 'it works on my machine' problems.



Common Beginner Mistakes & How to Fix Them

PROBLEM 1: ModuleNotFoundError: No module named 'numpy'

Solution: You forgot to activate your .venv. Run:

```
source .venv/bin/activate
```

(Mac/Linux) or

```
.venv\Scripts\activate
```

(Windows)

PROBLEM 2: pip: command not found

Solution: Python isn't in your PATH. Reinstall Python and check "Add Python to PATH" during installation.

PROBLEM 3: "Kernel not found" or "No kernel selected"

Solution: In VS Code, click the kernel selector (top right) and choose "Python (.venv)". In Antigravity, this is automatic.

PROBLEM 4: "(.venv) doesn't appear in my terminal"

Solution: You're in the wrong folder. Navigate to your project folder first: `cd my_project` then activate.

 Stuck? Ask your AI agent! Describe the error message and it will help you debug. Most beginner errors are simple fixes.